



长沙航空职业技术学院
Changsha Aeronautical Vocational and Technical College

毕 业 设 计 成 果

物化产品

设计题目： 电子式飞机气压空速表的设计与开发

二级学院 电子学院

专 业 飞机电子设备维修

班 级 飞电 2205 班

学 号 2022*****

姓 名 胡凯旋

指导教师 宋 焘



诚信声明

本人郑重声明：所呈交的毕业设计，是本人在老师的指导下，独立完成所取得的成果。尽我所知，设计中除特别加以标注的地方外，不包含其他人已经发表或撰写过的研究成果。本声明的法律结果由本人承担。

学生签名：

胡凯旋

2024年12月31日

指导教师签名：

(Signature)

2024年12月31日

目录

1 电子式飞机气压空速表的设计与开发任务与思路	1
1.1 设计目的	1
1.2 设计要求	2
1.3 设计思路	2
2 电子式飞机气压空速表电路的方案与选用	3
2.1 传感器电路方案与选用	3
2.2 数据处理电路的方案与选用	4
2.3 显示电路的方案与选用	4
2.4 电源电路的方案与选择	5
2.5 电子式飞机气压空速表系统硬件原理图	7
3 电子式飞机气压空速表电路的设计	8
3.1 传感器电路的设计	8
3.2 数据处理电路的设计	9
3.3 显示电路的设计	9
3.4 电源电路的设计	10
3.5 电子式飞机气压空速表整机电路图	10
4 电子式飞机气压空速表软件设计	11
4.1 主程序	11
4.2 子程序	11
5 电子式飞机气压空速表的安装与调试	12
5.1 产品安装	12
5.2 产品调试	13

总 结	14
-----	----

参考文献	15
------------	----

附录 1 主程序源代码	16
-------------------	----

附录 2 BMP180 传感器子程序源代码	19
-----------------------------	----

附录 3 AGR12 传感器子程序源代码	27
----------------------------	----

附录 4 0.96 寸 OLED 显示子程序源代码	32
---------------------------------	----

1 电子式飞机气压空速表的设计与开发任务与思路

1.1 设计目的

1.1.1 提高空速测量精度与可靠性

传统的飞机气压空速表在测量过程中可能受到多种因素的干扰，如机械部件的磨损、温度变化对气压传感器的影响等，导致测量精度下降和可靠性降低。通过电子式设计，利用先进的传感器技术和数字信号处理算法，能够有效减少这些干扰因素的影响，实现更精确、稳定的空速测量。

1.1.2 提供直观清晰的数字显示界面

相较于传统指针式气压空速表，数码显示方式能够以直观的数字形式直接呈现空速信息，避免了飞行员读取指针位置时可能产生的视觉误差和误判。飞行员在飞行过程中需要快速、准确地获取空速数据，以做出各种飞行决策，如起飞、降落、巡航速度调整等。电子式数码显示飞机气压空速表的数字界面设计可以根据人机工程学原理进行优化，使显示的数字字体大小、颜色、对比度等都符合飞行员在不同飞行环境下的视觉需求，确保空速信息一目了然，极大地提高了飞行员获取信息的效率和准确性，从而提升飞行操作的安全性与便捷性。

1.1.3 实现多功能集成与数据处理

现代飞机飞行需要处理和分析大量的飞行数据，单纯的空速测量已不能满足复杂飞行任务的需求。设计开发电子式数码显示飞机气压空速表的目的之一是实现多功能集成，使其不仅仅是一个空速测量仪器，还能够与飞机上的其他航空电子系统进行数据交互和融合。

1.1.4 便于系统维护与升级

随着航空技术的不断发展，飞机的航空电子设备需要不断进行维护和升级以适应新的飞行标准和要求。电子式数码显示飞机气压空速表采用模块化的设计理念，各个功能模块相对独立又相互协作，使得在设备出现故障时，能够快速定位和更换故障模块，降低维护成本和维修时间，提高飞机的出勤率。此外，这种数字化的设计结构也为后续的功能升级提供了便利，只需通过软件更新或更换部分硬件模块，就可以轻松实现新功能的添加或性能的提升，如适应新型飞机型号的特殊空速测量要求、集成更先进的航空电子通信协议等，确保空速表在飞机的整个使用寿命周期内都能保持良好的性能和兼容性，紧跟航空技术发展的步伐。

1.1.5 符合航空电子设备数字化发展趋势

当今航空领域正朝着数字化、智能化的方向飞速发展，飞机的各个系统都在逐步实现数字化改造和升级。设计电子式数码显示飞机气压空速表是顺应这一发展趋势的必然要求，它能够更好地与飞机上其他数字化航空电子设备进行无缝对接和协同工作，形成一个高效、智能的航空电子系统网络。这种数字化集成不仅

提高了单个设备的性能，更提升了整个飞机航空电子系统的整体效能和可靠性，为未来实现更高级的飞行自动化、智能化奠定了坚实的基础，促进了航空技术的持续进步和创新发展。

1.2 设计要求

1. 能运用所学的电工技术、模拟电子技术、数字电子技术、传感器技术、单片机应用技术和飞机仪表技术对飞机气压空速表进行整体方案设计；
2. 能根据整体设计方案正确选择合适的器件和电路；
3. 能制作硬件电路；
4. 能简单编程；
5. 能实现其基本功能。

1.3 设计思路

电子式飞机气压空速表的设计，首先通过传感器电路采集信号，然后将采集到的信号传输到数据处理电路进行信号处理，最后将处理好的信号通过显示电路显示出来。所以电子式飞机气压空速表由传感器电路、数据处理电路、显示电路、电源电路四部分电路组成。系统框图如图 1-1 所示。

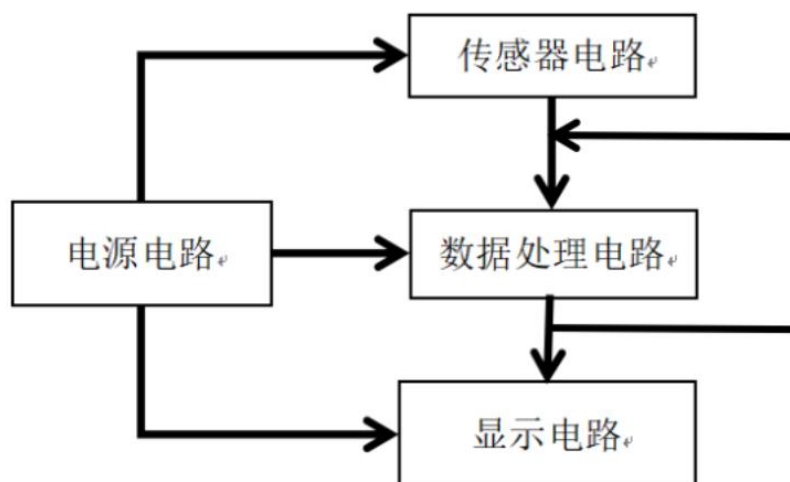


图 1-1 系统框图

2 电子式飞机气压空速表电路的方案与选用

2.1 传感器电路方案与选用

2.1.1 基于压敏电阻的方案

原理：压敏电阻的电阻值会随其所受气压变化而改变。将压敏电阻接入惠斯通电桥电路中，当气压变化导致压敏电阻阻值变化时，电桥的输出电压会产生相应的变化。这个微小的电压变化经过放大电路进行放大处理，以提高信号的幅值便于后续处理。然后通过滤波电路去除放大过程中引入的噪声干扰，得到较为纯净的与气压相关的电压信号。最后将该信号输入到模数转换器（ADC）中，将模拟电压信号转换为数字信号，以供微处理器进行空速计算等处理。

优点：压敏电阻价格相对较低，电路结构相对简单，易于实现和调试。在一定的气压测量范围内能够提供较为稳定的输出信号。

缺点：在对空速测量精度要求极高的飞机应用场景中满足较低。

2.2.2 基于电容式压力传感器的方案

原理：电容式压力传感器利用了电容值随压力变化而改变的特性。通常采用两个平行的电极板，其中一个电极板为可动膜片，当气压作用在膜片上时，膜片发生形变，导致两极板间的距离改变，从而使电容值发生变化。通过电容检测电路将电容值的变化转换为电压或频率信号的变化。例如采用电容 - 电压转换电路将电容变化转换为电压信号，然后经过放大、滤波处理后再由 ADC 转换为数字信号。或者采用电容 - 频率转换电路将电容变化转换为频率信号，这种方式对噪声有一定的抑制作用，后续通过计数器对频率进行测量并转换为数字量供微处理器处理。

优点：具有较高的精度和灵敏度，并且可以通过一些电路设计和校准手段进一步提高其在不同环境下的性能。

缺点：电容式压力传感器的电路相对复杂，设计和调试难度较大。传感器的成本较高，相比压敏电阻方案在价格上不占优势。

根据以上两种方案的对比且飞机气压空速表的设计要求对精度和测量稳定有很高的要求，方案二检测电路相对复杂，设计和调试难度较大，传感器的成本较高，不适合作为本设计的传感器，故采用方案一基于压敏电阻的方案。基于压敏电阻气压传感器实物如图 2-1



图 2-1 基于压敏电阻气压传感器实物

2.2 数据处理电路的方案与选用

数据处理电路在电子式飞机气压空速表中, 主要对传感器数据进行处理, 再将数据转化成数字信号进行处理显示在液晶屏上。常用的数据处理电路主要有两种。

方案一：利用单片机进行数据处理。单片机是一种可通过编程控制的微处理器，内部有寄存器，控制器，输入/输出接口。实际上是一个简单却又完整的计算机系统，它的功能的实现是要靠我们使用者编程来完成的。编程的目的就是控制这块芯片各个引脚在不同时间输出不同的电平，进而控制与单片机各个引脚相连的外围电路的电气状态。

优点：集成度高，系统结构简单，使用方便，实现模块化，工作可靠性高，处理功能强、速度快。

缺点：采集数据比较麻烦，需要大量的编程工作。

方案二：运用集成芯片等原件组成数据处理电路。集成芯片是现代数字集成芯片主要使用 CMOS 工艺制造的。CMOS 器件的静态功耗很低，但是在高速开关的情况下，CMOS 器件需要电源提供瞬时功率，高速 CMOS 器件的动态功率要求超过同类双极性器件。因此必须对这些器件加去耦电容以满足瞬时功率要求。

优点：抗干扰能力强。

缺点：数据处理复杂，集成化低，电路结构复杂。

由于电路需要数据处理信号较多,方案一单片机数据处理电路结构简单,而且能快速处理数字信号,方案二数据处理复杂,故采用方案一作为数据处理电路。常用的单片机型号有很多种,本仪器传感器较少数据处理要求不多故采用STC89C52RC 型号单片机,该单片机应用最广泛的 8 位单片机,编程较为简单,运算能力强稳定性高,价格也较为低廉。STC89C52RC 引脚图如图 2-2 所示。

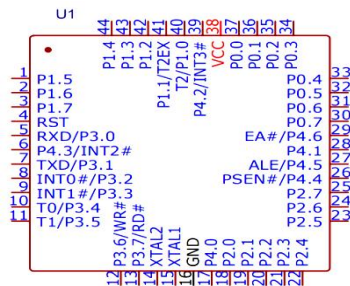


图 2-2 STC89C52RC 单片机引脚图

2.3 显示电路的方案与选用

显示电路在电子式飞机气压空速表中负责将主控制电路处理好的数据通过显示系统以数字的形式显示出来，能让机长直观地了解飞机气压空速表的情况。常用的显示电路方案有以下两种。

方案一：利用 0.96 寸 OLED 液晶进行显示。0.96 寸 OLED 是一种具有 4 位/8

位并行、2 线或 3 线串行拟多种接口方式，内部含有国标一级、二级简体中文字库的点阵接口方式和简单、方便的操作指令，低功耗是其又一显著特点。由该模块构成的液晶方案与同类型的点阵液晶显示模块相比，不论硬件电路结构或显示程序都要简洁很多，且该模块的价格也略低于相同点阵的图形液晶模块。

优点：显示内容丰富；

缺点：易损坏；

方案二：利用数码管进行显示。数码管由多个发光二极管封装在一组成 8 字型的器件，引线已在内部连接完成，只需引出他们的各个比划，公共电极。多位数码管通过分时轮流控制各个 LED 数码管的 COM 端，就使各个数码管轮流受控制显示，这就是动态驱动。每位数码管点亮的时间为 1-2ms，由于人的视觉暂留现象及发光二极管的余辉效应，实际上各位数码管并非同时点亮，但只要扫描的速度够快，给人的印象就是一组稳定的显示不会有闪烁感，动态显示的效果和静态显示的效果是一样的，能够节省大量的 I/O 口，而且功耗更低。

优点：功耗低，编程简单，价格低

缺点：显示内容简单。

电子式飞机气压空速表，未来的飞机上液晶屏幕占主导，飞机各种数据只需集成正在一个液晶屏幕上，显然数码管不能满足要求，选择作为显示 0.96 寸 OLED 屏显示，实物如图 2-3 所示。

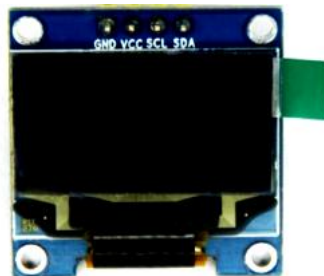


图 2-3 0.96 寸 OLED 液晶屏

2.4 电源电路的方案与选择

2.4.1 电源电路方案

1. 开关稳压电源电路方案

原理：

开关稳压电源电路主要由输入整流滤波电路、开关控制器、高频变压器、输出整流滤波电路和反馈电路组成。首先，输入整流滤波电路将飞机电源（交流或直流）转换为直流并进行初步滤波。然后，开关控制器通过控制开关管的导通和截止时间，将直流电压转换为高频脉冲电压。高频变压器用于隔离输入和输出，并根据需要调整电压幅值。输出整流滤波电路将高频脉冲电压转换为稳定的直流电压供给负载。反馈电路监测输出电压，并将信号反馈给开关控制器，以调整开

关管的导通和截止时间，从而实现输出电压的精确控制。

优点：

效率高，它可以方便地实现多种输出电压，通过调整高频变压器的匝数比和开关控制器的参数，能够满足空速表中不同模块对不同电压的需求。

缺点：

电路结构复杂，涉及到高频电路和复杂的控制环路，设计和调试难度较大。其输出电压纹波相对线性稳压电源可能会稍大一些，不过通过合理的电路设计和滤波措施可以将纹波控制在可接受的范围内。另外，开关稳压电源在工作过程中会产生电磁干扰，需要采取适当的 EMI 抑制措施，如添加屏蔽罩、使用滤波电感和电容等，以避免对飞机上其他电子设备造成干扰。

2. 电池备份电源电路方案

原理：

除了飞机主电源外，设计电池备份电源电路。该电路主要包括可充电电池（如锂电池）、电池充电管理电路和电池切换电路。在飞机主电源正常供电时，电池充电管理电路对电池进行充电，使其保持满电状态。当主电源出现故障或波动超出允许范围时，电池切换电路会迅速将电源切换到电池供电模式，确保空速表能够继续正常工作。电池输出的电压可能需要经过适当的稳压电路（如线性稳压或开关稳压）来满足空速表的电压要求。

优点：

提供了电源备份功能，增强了空速表的可靠性和应急能力。在飞机主电源故障的情况下，能够维持空速表的基本功能，为飞行员提供关键的空速信息，保障飞行安全。而且电池备份电源可以在飞机启动和关机阶段，为一些需要初始化或保存数据的电路模块提供稳定的电源支持。

缺点：

需要额外的空间来安装电池和相关的充电管理、切换电路。电池的使用寿命有限，需要定期维护和更换，因此需要严格的电池管理和安全措施。

2.4.2 电源电路选择考虑因素

1. 电源效率

如果飞机对能源利用效率要求较高，或者空速表需要长期稳定工作而飞机电源供应相对紧张，开关稳压电源是较好的选择。它能够在不同的输入电压和负载条件下保持较高的效率，减少能源浪费。但如果对电源效率要求不是特别高，且电路简单性和稳定性更为重要，线性稳压电源也可以满足基本需求。

2. 空间限制

在飞机仪表空间有限的情况下，开关稳压电源的小体积优势明显。它可以在

不占用过多空间的情况下提供稳定的电源。而线性稳压电源由于其较大的变压器和滤波元件，可能会受到空间限制。电池备份电源也需要考虑电池和相关电路的安装空间，不过在可靠性要求较高的情况下，即使空间有限，也可能需要考虑采用电池备份电源。

2.4.3 对比后选择

把两种电源电路方案比较，结合电子式飞机气压空速表在设计初期的需要，可蓄电池更适合作为供电电源，后期可根据应用更改供电方式。蓄电池种类多，为了方便携带和环保，选择锂电池作为电源电路进行供电。锂电池实物如图 2-4 所示。



图 2-4 锂电池图

2.5 电子式飞机气压空速表系统硬件原理图

压力传感器采集数据，将数据信息传给 STC89C52RC 单片机进行信号处理，将处理好的信息在 0.96 寸 OLED 液晶屏幕上显示。整机原理图如图 2-5 所示。

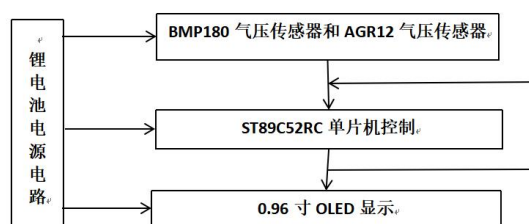


图 2-5 结构框图

3 电子式飞机气压空速表电路的设计

3.1 传感器电路的设计

电子式飞机气压空速表需要将 BMP180 气压传感器测得的温度和静压的数据和 AGR12 气压传感器测得的动压的数据一起传输到单片机数据处理电路进行处理。将 BMP180 气压传感器和单片机 I/O 口进行连接。1 脚为接地脚，2 脚为电源脚，3 脚为时钟脚，4 脚为数据脚。将 AGR12 气压传感器和单片机 I/O 口进行连接。1 脚为接地脚，2 脚为电源脚，3 脚为时钟脚，4 脚为数据脚。气压传感器模块电路图如图 3-1、3-2，实物如图 3-3、3-4 所示。

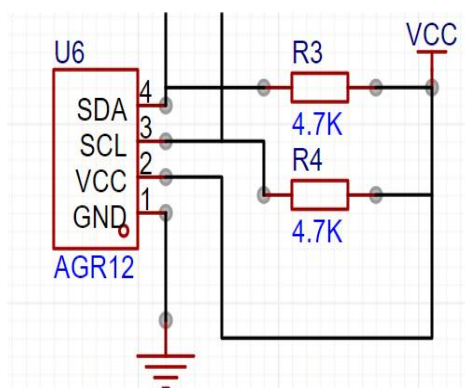


图 3-1 BMP180 气压传感器电路图

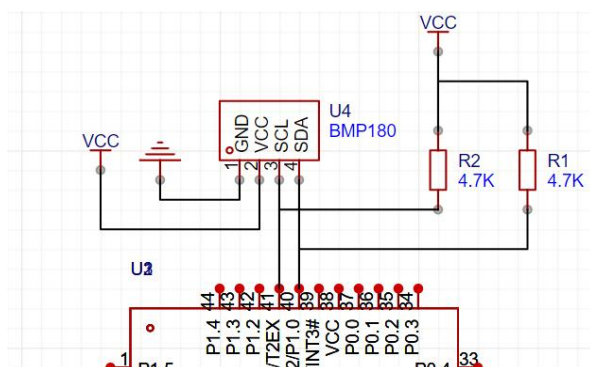


图 3-2 AGE12 气压传感器电路图



图 3-3 BMP180 气压传感器实物图

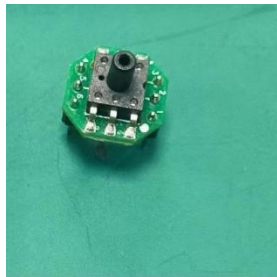


图 3-4 AGR12 气压传感器实物图

3.2 数据处理电路的设计

数据处理电路主要为单片机最小系统，包括电源电路、晶振电路、复位电路。电源由锂电池提供 5V 直流电供单片机工作；晶振电路由 12MHz 和两个并联的 33pF 电容组成；复位电路是由电容串联电阻构成。数据处理电路如图 3-5。数据处理电路实物图如图 3-6。

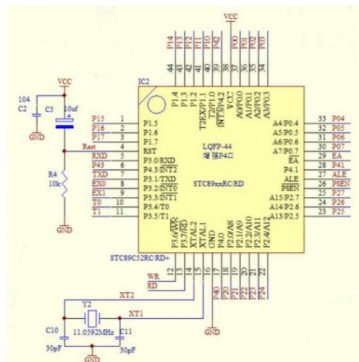


图 3-7 显示电路图

3.4 电源电路的设计

电子式飞机气压空速表中,将锂电池通过迷你 USB 电源头连接和自锁开关控制电源电路开关给各单元电路供电。如图 3-8 所示。

图 3-8 电源电路图

3.5 电子式飞机气压空速表整机电路图

电子式飞机气压空速表整机电路图如图 3-9 所示。

图 3-9 整机电路原理图

4 电子式飞机气压空速表软件设计

4.1 主程序

电子式飞机气压空速表主程序的设计思路是 STC89C51 开始,将 0.96 寸 OLED 液晶屏幕和气压传感器所需的程序初始化,然后接收由气压电路返回的气压数据,最后将收集到的数据进行处理通过显示电路显示。整个过程将持续更新数据,主程序见附录,主程序流程图如图 4-1 所示。

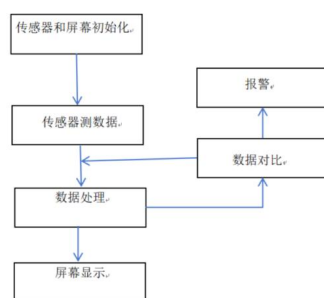


图 4-1 主程序框图

4.2 子程序

4.2.1 传感器子程序

BMP180 和 AGR12 在初始化之后,将发送指令使气压感器测量温度、静压和动压,收到指令后传感器电路将测温度、静压和动压,将测到的静压和动压数据传输数据处理电路。传感器子程序见附录,传感器子程序流程图如图 4-2 所示。

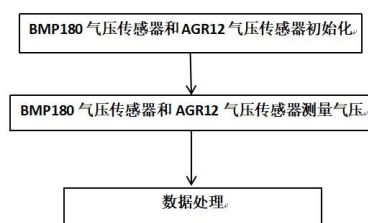


图 4-2 传感器子程序框图

4.2.2 显示子程序

0.96 寸 OLED 液晶显示首先将初始化序列,然后设置屏幕坐标,最后开启显示。子程序见附录 3,显示子程序流程图如图 4-3 所示。

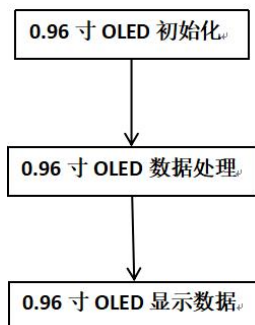


图 4-3 显示子程序框图

5 电子式飞机气压空速表的安装与调试

5.1 产品安装

5.1.1 产品安装元器件清单

产品安装元器件清单如表 5-1 所示。

表 5-1 产品安装元器件清单

元件名称	数量	单位
最小系统板	1	套
BMP180 气压传感器	1	个
AGR12 气压传感器	1	个
0.96 寸 OLED	1	个
杜邦线	12	根
锂电池	1	个

5.1.2 产品安装注意事项

- (1) 电源切勿短接；
- (2) 单片机芯片不要装反；
- (3) 液晶屏、超声波引脚不要装错；
- (4) 不能带电操作；
- (5) 焊接时注意焊点不要虚焊、连锡。

5.1.3 产品安装成果

电子式飞机气压空速表实物图如图 5-1 所示。

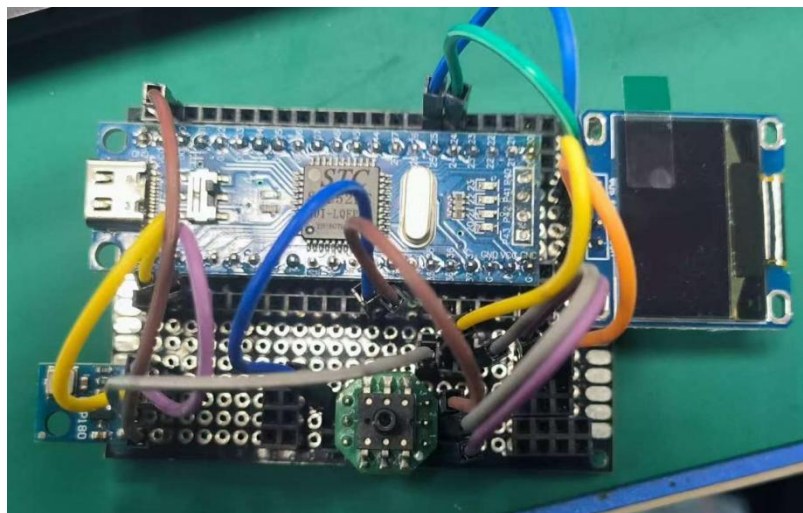


图 5-1 电子式飞机气压空速表实物图

5.2 产品调试以及成果展示

5.2.1 测试空速

用打气桶模拟飞机飞行时的动压。测试效果如图 5-2 所示。

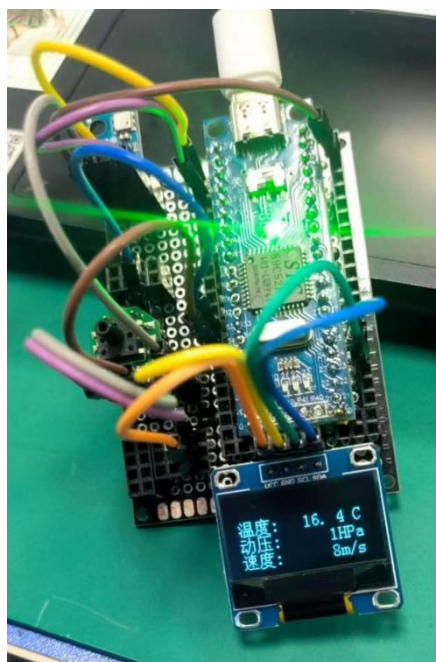


图 5-2 检测空速效果图

5.2.2 测试空速报警

用打气桶模拟飞机飞行时的动压。当飞机飞行速度过高，测试效果如图 5-3 所示。



图 5-3 检测空速报警效果图

总 结

该设计的目的是设计一个电子式飞机气压空速表的设计与开发展开,在设计目的方面,旨在提高空速测量精度与可靠性,以数字显示提供直观清晰界面,实现多功能集成与数据处理,便于系统维护升级,契合航空电子设备数字化发展趋势。设计要求涉及多方面技术运用及功能实现。

设计思路,空速表由传感器、数据处理、显示和电源电路组成。传感器电路经对比选用基于压敏电阻方案;数据处理电路采用 STC89C52RC 单片机;显示电路选择 0.96 寸 OLED 屏;电源电路综合考虑后确定锂电池供电。

硬件设计涵盖各电路设计与连接,软件设计包括主程序及传感器、显示等子程序流程。产品安装需注意多项事项,调试通过打气桶模拟动压测试空速及报警功能,实物图展示了最终成果。该研究为飞机气压空速表设计提供了全面的理论与实践参考,有助于推动航空电子设备技术发展。

在电子式飞机气压空速表设计中,用到了电工电子技术,单片机、传感器、飞机仪表等课程知识。这些知识对此次设计起到重要作用,通过此次设计在电路设计上有很大提升。毕业设计将进一步巩固加深理论知识和专业技能。

最后,感谢老师的辛苦指导,祝老师身体健康,工作顺利!

参考文献

- [1] 李华. 现代电工技术基础与实践[M]. 北京: 机械工业出版社, 2022.
- [2] 王强. 电子技术实用教程[M]. 北京: 中国电力出版社, 2023.
- [3] 张宏. 单片机编程实战指南[M]. 北京: 科学出版社, 2022.
- [4] 陈杰. 单片机 C51 编程从入门到精通[M]. 北京: 清华大学出版社, 2021.
- [5] 赵亮. 传感器技术原理与应用新解[M]. 北京: 北京大学出版社, 2023.
- [6] 钱峰. 飞机仪表原理与系统[M]. 北京: 航空工业出版社, 2021.



附录 1 主程序源代码

```
/*  
*****  
*****  
项目名称：电子式飞机气压空速表  
项目人员：胡凯旋  
指导老师：宋烨  
设计时间：2024 年 12 月 23 日  
*/  
*****  
  
#include <STC89C5xRC.H>  
#include "OLED.h"  
#include "DELAY1.h"  
#include "BMP180IIC.h"  
#include "AGR12YX.h"  
#include "math.h"  
  
  
#define R 8.314  
#define M 0.02897  
  
void Get_Dat();  
  
void main()  
{  
    u32 v, q, p1, k, v1, q1, k1, k2;  
  
    init_uart();  
    Init_BMP180();  
    OLED_Init();  
    OLED_Clear();  
    while (1)  
    {  
        q=read_pressure1();
```



```
q1=q;
p1=bmp180Convert (BMP180_GET_PRESSURE)*10;
k=bmp180Convert (BMP180_GET_TEMP)/10;
k2=bmp180Convert (BMP180_GET_TEMP)%10;
k1=k+273;
v=sqrt((2 * q1 * R * k1 / (p1 * M)));
v1=v/10;
if(v1<=230)
{

    OLED_ShowCHinese(0,2,0);
    OLED_ShowCHinese(16,2,1);
    OLED_ShowString(32, 2, ":", 16);
    OLED_ShowNum(64,2,k,2,16);
    OLED_ShowString(80, 2, ".", 16);
    OLED_ShowNum(96,2,k2,1,16);
    OLED_ShowString(112, 2, "C", 16);

    OLED_ShowCHinese(0,4,2);
    OLED_ShowCHinese(16,4,3);
    OLED_ShowString(32, 4, ":", 16);
    OLED_ShowNum(48,4,q1/1000,6,16);
    OLED_ShowString(96, 4, "hPa", 16);

    OLED_ShowCHinese(0,6,4);
    OLED_ShowCHinese(16,6,1);
    OLED_ShowString(32, 6, ":", 16);
    OLED_ShowNum(48,6,v1,6,16);
    OLED_ShowString(96,6, "m/s", 16);
}
else
{
    OLED_ShowCHinese(0,0,5);
    OLED_ShowCHinese(32,0,6);
```



```
OLED_ShowCHinese(0, 2, 0);
OLED_ShowCHinese(16, 2, 1);
OLED_ShowString(32, 2, ":", 16);
OLED_ShowNum(64, 2, k, 2, 16);
OLED_ShowString(80, 2, ". ", 16);
OLED_ShowNum(96, 2, k2, 1, 16);
OLED_ShowString(112, 2, "C", 16);

OLED_ShowCHinese(0, 4, 2);
OLED_ShowCHinese(16, 4, 3);
OLED_ShowString(32, 4, ":", 16);
OLED_ShowNum(48, 4, q1/1000, 6, 16);
OLED_ShowString(96, 4, "hPa", 16);

OLED_ShowCHinese(0, 6, 4);
OLED_ShowCHinese(16, 6, 1);
OLED_ShowString(32, 6, ":", 16);
OLED_ShowNum(48, 6, v1, 6, 16);
OLED_ShowString(96, 6, "m/s", 16);

    }
    delay(1);
}
}
```

附录2 BMP180 传感器子程序源代码

```
/******  
*****  
项目名称：电子式飞机气压空速表  
项目人员：胡凯旋  
指导老师：宋烨  
设计时间：2024 年 12 月 23 日  
/******  
*****  
#include <STC89C5xRC.H>  
#include "BMP180IIC.h"  
#include "DELAY1.h"  
  
void Single_Write(uchar SlaveAddress,uchar REG_Address,uchar  
REG_data);  
uchar Single_Read(uchar REG_Address);  
void Multiple_Read(uchar,uchar);  
  
void BMP180_Start();  
void BMP180_Stop();  
void BMP180_SendACK(bit ack);  
bit BMP180_RecvACK();  
void BMP180_SendByte(uchar dat);  
uchar BMP180_RecvByte();  
void BMP180_ReadPage();  
  
void BMP180_Start()  
{  
    SDA = 1;  
    SCL = 1;  
    Delay5us();  
    SDA = 0;  
    Delay5us();  
    SCL = 0;
```



```
}
```

```
void BMP180_Stop()
```

```
{
```

```
    SDA = 0;
```

```
    SCL = 1;
```

```
    Delay5us();
```

```
    SDA = 1;
```

```
    Delay5us();
```

```
}
```

```
void BMP180_SendACK(bit ack)
```

```
{
```

```
    SDA = ack;
```

```
    SCL = 1;
```

```
    Delay5us();
```

```
    SCL = 0;
```

```
    Delay5us();
```

```
}
```

```
bit BMP180_RecvACK()
```

```
{
```

```
    SCL = 1;
```

```
    Delay5us();
```

```
    CY = SDA;
```

```
    SCL = 0;
```

```
    Delay5us();
```

```
    return CY;
```

```
}
```



```
void BMP180_SendByte(uchar dat)
{
    uchar i;

    for (i=0; i<8; i++)
    {
        dat <<= 1;
        SDA = CY;
        SCL = 1;
        Delay5us();
        SCL = 0;
        Delay5us();
    }
    BMP180_RecvACK();
}
```

```
uchar BMP180_RecvByte()
{
    uchar i;
    uchar dat = 0;

    SDA = 1;
    for (i=0; i<8; i++)
    {
        dat <<= 1;
        SCL = 1;
        Delay5us();
        dat |= SDA;
        SCL = 0;
        Delay5us();
    }
    return dat;
}
```



}

```
short Multiple_read(uchar ST_Address)
{
    uchar msb, lsb;
    short _data;
    BMP180_Start();
    BMP180_SendByte(BMP180_SlaveAddress);
    BMP180_SendByte(ST_Address);
    BMP180_Start();
    BMP180_SendByte(BMP180_SlaveAddress+1);
    msb = BMP180_RecvByte();
    BMP180_SendACK(0);
    lsb = BMP180_RecvByte();
    BMP180_SendACK(1);

    BMP180_Stop();
    Delay5ms();
    _data = msb << 8;
    _data |= lsb;
    return _data;
}
```

```
long bmp180ReadTemp(void)
{
    BMP180_Start();
    BMP180_SendByte(BMP180_SlaveAddress);
    BMP180_SendByte(0xF4);
    BMP180_SendByte(0x2E);
    BMP180_Stop();
    delay(10); // max time is 4.5ms

    return (long) Multiple_read(0xF6);
}
```



```
}

long bmp180ReadPressure(void)
{
    long pressure = 0;

    BMP180_Start();
    BMP180_SendByte(BMP180_SlaveAddress);
    BMP180_SendByte(0xF4);
    BMP180_SendByte(0x34);
    BMP180_Stop();
    delay(10);

    pressure = Multiple_read(0xF6);
    pressure &= 0x0000FFFF;

    return pressure;
}

void Init_BMP180()
{
    short ac1;
    short ac2;
    short ac3;
    unsigned short ac4;
    unsigned short ac5;
    unsigned short ac6;
    short b1;
    short b2;
    short mb;
    short mc;
    short md;
    ac1 = Multiple_read(0xAA);
```



```
ac2 = Multiple_read(0xAC);
ac3 = Multiple_read(0xAE);
ac4 = Multiple_read(0xB0);
ac5 = Multiple_read(0xB2);
ac6 = Multiple_read(0xB4);
b1 = Multiple_read(0xB6);
b2 = Multiple_read(0xB8);
mb = Multiple_read(0xBA);
mc = Multiple_read(0xBC);
md = Multiple_read(0xBE);
}

//*****
*****

long bmp180Convert(uchar Type)
{
    long ut;
    long up;
    long x1, x2, b5, b6, x3, b3, p;
    unsigned long b4, b7;
    long Dat_Tem;

    switch(Type)
    {
        case BMP180_GET_TEMP:
        {
            ut = bmp180ReadTemp();
            ut = bmp180ReadTemp();
            //*****
            x1 = (ut - Multiple_read(0xB4)) * Multiple_read(0xB2) >> 15;
            x2 = ((long)Multiple_read(0xBC)<< 11) / (x1 +
Multiple_read(0xBE));
            b5 = x1 + x2;
            Dat_Tem = (b5 + 8) >> 4;
```



```
        break;
    }

    case BMP180_GET_PRESSURE:
    {
        up = bmp180ReadPressure();
        up = bmp180ReadPressure();
        //*****
        b6 = b5 - 4000;
        x1 = ( Multiple_read(0xB8) * (b6 * b6 >> 12)) >> 11;
        x2 = Multiple_read(0xAC) * b6 >> 11;
        x3 = x1 + x2;
        b3 = (((long)Multiple_read(0xAA) * 4 + x3) + 2)/4;
        x1 = Multiple_read(0xAE) * b6 >> 13;
        x2 = (Multiple_read(0xB6) * (b6 * b6 >> 12)) >> 16;
        x3 = ((x1 + x2) + 2) >> 2;
        b4 = (Multiple_read(0xB0) * (unsigned long) (x3 + 32768)) >>
15;

        b7 = ((unsigned long) up - b3) * (50000 >> 0SS);
        if( b7 < 0x80000000)
            p = (b7 * 2) / b4 ;
        else
            p = (b7 / b4) * 2;
        x1 = (p >> 8) * (p >> 8);
        x1 = (x1 * 3038) >> 16;
        x2 = (-7357 * p) >> 16;
        Dat_Tem = (p + ((x1 + x2 + 3791) >> 4));

        break;
    }

    default:
        break;
}
```

```
    return Dat_Tem;  
}
```

附录3 AGR12 传感器子程序源代码

```
/******  
*****  
项目名称：电子式飞机气压空速表  
项目人员：胡凯旋  
指导老师：宋烨  
设计时间：2024 年 12 月 23 日  
/******  
*****  
#include <STC89C5xRC.H>  
#include "DELAY1.h"  
  
// I2C Pins  
sbit AGR12_SCL = P3^4;  
sbit AGR12_SDA = P3^2;  
  
void I2C_Start();  
void I2C_Stop();  
void I2C_SendByte(unsigned char byte);  
unsigned char I2C_ReceiveByte();  
void I2C_SendAck(bit ack);  
bit I2C_ReceiveAck();  
  
// I2C Start Condition  
void I2C_Start()  
{  
    AGR12_SDA = 1;  
    AGR12_SCL = 1;  
    delay(1);  
    AGR12_SDA = 0;  
    delay(1);
```



```
    AGR12_SCL = 0;
}

// I2C Stop Condition
void I2C_Stop()
{
    AGR12_SDA = 0;
    AGR12_SCL = 1;
    delay(1);
    AGR12_SDA = 1;
    delay(1);
}

// I2C Send Byte
void I2C_SendByte(unsigned char byte)
{
    unsigned char i;
    for (i = 0; i < 8; i++)
    {
        AGR12_SDA = (byte & 0x80) >> 7;
        AGR12_SCL = 1;
        delay(1);
        AGR12_SCL = 0;
        byte <<= 1;
    }
    I2C_ReceiveAck();
}

// I2C Receive Byte
unsigned char I2C_ReceiveByte()
{
    unsigned char i, byte = 0;
    AGR12_SDA = 1;
    for (i = 0; i < 8; i++)
```



```
{  
    AGR12_SCL = 1;  
    delay(1);  
    byte <<= 1;  
    if (AGR12_SDA)  
        byte++;  
    AGR12_SCL = 0;  
}  
return byte;  
}  
  
// I2C Send Acknowledge  
void I2C_SendAck(bit ack)  
{  
    AGR12_SDA = ack;  
    AGR12_SCL = 1;  
    delay(1);  
    AGR12_SCL = 0;  
}  
  
// I2C Receive Acknowledge  
bit I2C_ReceiveAck()  
{  
    bit ack;  
    AGR12_SDA = 1;  
    AGR12_SCL = 1;  
    delay(1);  
    ack = AGR12_SDA;  
    AGR12_SCL = 0;  
    return ack;  
}
```



```
#include <STC89C5xRC.H>
#include "AGR12IIC.h"
#include "DELAY1.h"

#define SENSOR_ADDR 0xA0

void init_uart();
int read_pressure1();

void init_uart()
{
    TMOD = 0x20; // Timer1 in Mode 2
    TH1 = 0xFD; // Baud rate 9600 for 11.0592 MHz
    TL1 = 0xFD;
    TR1 = 1; // Start Timer1
    SCON = 0x50; // UART Mode 1, REN enabled
    EA = 1; // Enable global interrupts
    ES = 1; // Enable serial interrupts
}

int read_pressure1()
{
    unsigned char data0, data1, data2;
    int pressure;

    // Send pressure measurement command
    I2C_Start();
    I2C_SendByte(SENSOR_ADDR);
    I2C_SendByte(0xAC);
    I2C_SendByte(0x12);
    I2C_Stop();
```



```
delay(80); // Wait for measurement to complete

// Read data from sensor
I2C_Start();
I2C_SendByte(SENSOR_ADDR | 0x01);
data0 = I2C_ReceiveByte();
I2C_SendAck(0);
data1 = I2C_ReceiveByte();
I2C_SendAck(0);
data2 = I2C_ReceiveByte();
I2C_SendAck(1);
I2C_Stop();

// Combine data bytes into pressure value
pressure = (data0 << 8) | data1;
if (pressure & 0x8000) // Check if negative
{
    pressure &= 0x7FFF;
    pressure = -pressure;
}
pressure = pressure*1000; // Convert to kPa

return pressure;
}
```

附录 4 0.96 寸 OLED 显示子程序源代码

```
/*  
*****  
项目名称：电子式飞机气压空速表  
项目人员：胡凯旋  
指导老师：宋烨  
设计时间：2024 年 12 月 23 日  
*/  
*****  
  
#include <STC89C5xRC.H>  
#include "DELAY1.h"  
#include "oled.h"  
  
#include "oledfont.h"  
  
void IIC_Start()  
{  
  
    OLED_SCLK_Set() ;  
    OLED_SDIN_Set() ;  
    OLED_SDIN_Clr() ;  
    OLED_SCLK_Clr() ;  
}  
  
void IIC_Stop()  
{  
    OLED_SCLK_Set() ;  
  
    OLED_SDIN_Clr() ;  
    OLED_SDIN_Set() ;  
  
}
```



```
void IIC_Wait_Ack()  
{
```

```
  
    OLED_SCLK_Set() ;  
    OLED_SCLK_Clr() ;  
}
```

```
  
void Write_IIC_Byte(unsigned char IIC_Byte)  
{
```

```
    unsigned char i;  
    unsigned char m, da;  
    da=IIC_Byte;  
    OLED_SCLK_Clr() ;  
    for(i=0;i<8;i++)  
    {
```

```
        m=da;
```

```
  
        m=m&0x80;  
        if(m==0x80)  
            {OLED_SDIN_Set();}  
        else OLED_SDIN_Clr();  
        da=da<<1;  
        OLED_SCLK_Set();  
        OLED_SCLK_Clr();  
    }
```

```
}
```

```
  
void Write_IIC_Command(unsigned char IIC_Command)
```

```
{  
    IIC_Start();
```



```
    Write_IIC_Byte(0x78);
    IIC_Wait_Ack();
    Write_IIC_Byte(0x00);
    IIC_Wait_Ack();
    Write_IIC_Byte(IIC_Command);
    IIC_Wait_Ack();
    IIC_Stop();
}

void Write_IIC_Data(unsigned char IIC_Data)
{
    IIC_Start();
    Write_IIC_Byte(0x78);
    IIC_Wait_Ack();
    Write_IIC_Byte(0x40);
    IIC_Wait_Ack();
    Write_IIC_Byte(IIC_Data);
    IIC_Wait_Ack();
    IIC_Stop();
}

void OLED_WR_Byte(unsigned dat,unsigned cmd)
{
    if(cmd)
    {

        Write_IIC_Data(dat);

    }
    else {
        Write_IIC_Command(dat);
    }
}
```



```
}

void fill_picture(unsigned char fill_Data)
{
    unsigned char m,n;
    for (m=0;m<8;m++)
    {
        OLED_WR_Byte(0xb0+m, 0);
        OLED_WR_Byte(0x02, 0);
        OLED_WR_Byte(0x10, 0);
        for (n=0;n<128;n++)
        {
            OLED_WR_Byte(fill_Data, 1);
        }
    }
}

void Delay_50ms(unsigned int Del_50ms)
{
    unsigned int m;
    for (;Del_50ms>0;Del_50ms--)
        for (m=6245;m>0;m--);
}

void Delay_1ms(unsigned int Del_1ms)
{
    unsigned char j;
    while(Del_1ms--)
    {
        for (j=0;j<123;j++);
    }
}
```



```
void OLED_Set_Pos(unsigned char x, unsigned char y)
{
    OLED_WR_Byte(0xb0+y, OLED_CMD);
    OLED_WR_Byte((((x+2)&0xf0)>>4)|0x10, OLED_CMD);
    OLED_WR_Byte(((x+2)&0x0f), OLED_CMD);
}

void OLED_Display_On(void)
{
    OLED_WR_Byte(0X8D, OLED_CMD);
    OLED_WR_Byte(0X14, OLED_CMD);
    OLED_WR_Byte(0XAF, OLED_CMD);
}

void OLED_Display_Off(void)
{
    OLED_WR_Byte(0X8D, OLED_CMD);
    OLED_WR_Byte(0X10, OLED_CMD);
    OLED_WR_Byte(0XAE, OLED_CMD);
}

void OLED_Clear(void)
{
    {
        u8 i,n;
        for(i=0;i<8;i++)
        {
            OLED_WR_Byte (0xb0+i, OLED_CMD);
            OLED_WR_Byte (0x02, OLED_CMD);
            OLED_WR_Byte (0x10, OLED_CMD);
            for(n=0;n<128;n++) OLED_WR_Byte (0, OLED_DATA);
        }
    }
}

void OLED_On(void)
{

```



```
u8 i, n;
for(i=0; i<8; i++)
{
    OLED_WR_Byte (0xb0+i, OLED_CMD);
    OLED_WR_Byte (0x02, OLED_CMD);
    OLED_WR_Byte (0x10, OLED_CMD);
    for(n=0; n<128; n++) OLED_WR_Byte(1, OLED_DATA);
}
}
```

```
void OLED_ShowChar(u8 x, u8 y, u8 chr, u8 Char_Size)
{
    unsigned char c=0, i=0;
    c=chr-' ';
    if(x>Max_Column-1) {x=0; y=y+2;}
    if(Char_Size ==16)
    {
        OLED_Set_Pos(x, y);
        for(i=0; i<8; i++)
            OLED_WR_Byte(F8X16[c*16+i], OLED_DATA);
        OLED_Set_Pos(x, y+1);
        for(i=0; i<8; i++)
            OLED_WR_Byte(F8X16[c*16+i+8], OLED_DATA);
    }
    else {
        OLED_Set_Pos(x, y);
        for(i=0; i<6; i++)
            OLED_WR_Byte(F6x8[c][i], OLED_DATA);
    }
}
```

```
u32 oled_pow(u8 m, u8 n)
{

```



```
    u32 result=1;
    while(n--)result*=m;
    return result;
}

void OLED_ShowNum(u8 x,u8 y,u32 num,u8 len,u8 size2)
{
    u8 t,temp;
    u8 enshow=0;
    for(t=0;t<len;t++)
    {
        temp=(num/oled_pow(10,len-t-1))%10;
        if(enshow==0&&t<(len-1))
        {
            if(temp==0)
            {
                OLED_ShowChar(x+(size2/2)*t,y,' ',size2);
                continue;
            }else enshow=1;
        }
        OLED_ShowChar(x+(size2/2)*t,y,temp+'0',size2);
    }
}

void OLED_ShowString(u8 x,u8 y,u8 *chr,u8 Char_Size)
{
    unsigned char j=0;
    while (chr[j]!='\0')
    {
        OLED_ShowChar(x,y,chr[j],Char_Size);
        x+=8;
        if(x>120){x=0;y+=2;}
        j++;
    }
}
```



```
}
```

```
void OLED_ShowCHinese(u8 x,u8 y,u8 no)
{
    u8 t, adder=0;
    OLED_Set_Pos(x, y);
    for(t=0;t<16;t++)
    {
        OLED_WR_Byte(Hzk[2*no][t], OLED_DATA);
        adder+=1;
    }
    OLED_Set_Pos(x, y+1);
    for(t=0;t<16;t++)
    {
        OLED_WR_Byte(Hzk[2*no+1][t], OLED_DATA);
        adder+=1;
    }
}
```

```
void OLED_DrawBMP(unsigned char x0, unsigned char y0,unsigned char
x1, unsigned char y1,unsigned char BMP[])
{
    unsigned int j=0;
    unsigned char x,y;

    if(y1%8==0) y=y1/8;
    else y=y1/8+1;
    for(y=y0;y<y1;y++)
    {
        OLED_Set_Pos(x0, y);
        for(x=x0;x<x1;x++)
        {
            OLED_WR_Byte(BMP[j++], OLED_DATA);
        }
    }
}
```



```
}  
  
}  
  
  
void OLED_Init(void)  
{  
  
    OLED_WR_Byte(0xAE, OLED_CMD); //--display off  
    OLED_WR_Byte(0x00, OLED_CMD); //---set low column address  
    OLED_WR_Byte(0x10, OLED_CMD); //---set high column address  
    OLED_WR_Byte(0x40, OLED_CMD); //--set start line address  
    OLED_WR_Byte(0xB0, OLED_CMD); //--set page address  
    OLED_WR_Byte(0x81, OLED_CMD); // contract control  
    OLED_WR_Byte(0xFF, OLED_CMD); //--128  
    OLED_WR_Byte(0xA1, OLED_CMD); //set segment remap  
    OLED_WR_Byte(0xA6, OLED_CMD); //--normal / reverse  
    OLED_WR_Byte(0xA8, OLED_CMD); //--set multiplex ratio(1 to 64)  
    OLED_WR_Byte(0x3F, OLED_CMD); //--1/32 duty  
    OLED_WR_Byte(0xC8, OLED_CMD); //Com scan direction  
    OLED_WR_Byte(0xD3, OLED_CMD); //-set display offset  
    OLED_WR_Byte(0x00, OLED_CMD); //  
  
    OLED_WR_Byte(0xD5, OLED_CMD); //set osc division  
    OLED_WR_Byte(0x80, OLED_CMD); //  
  
    OLED_WR_Byte(0xD8, OLED_CMD); //set area color mode off  
    OLED_WR_Byte(0x05, OLED_CMD); //  
  
    OLED_WR_Byte(0xD9, OLED_CMD); //Set Pre-Charge Period  
    OLED_WR_Byte(0xF1, OLED_CMD); //  
  
    OLED_WR_Byte(0xDA, OLED_CMD); //set com pin configuration  
    OLED_WR_Byte(0x12, OLED_CMD); //
```



```
OLED_WR_Byte(0xDB, OLED_CMD); //set Vcomh  
OLED_WR_Byte(0x30, OLED_CMD); //  
  
OLED_WR_Byte(0x8D, OLED_CMD); //set charge pump enable  
OLED_WR_Byte(0x14, OLED_CMD); //  
  
OLED_WR_Byte(0xAF, OLED_CMD); //--turn on oled panel  
}
```